

Week 11 迁移学习&GAN 网络 思考题

02L

01.根据视频《093 迁移学习代码实现 01》中所讲述的内容，完成视频中的训练过程实验。

02.根据上题完成的实验，比较解冻不同层数的参数时，训练结果会有什么变化

03.将第 01 题中完成的流程应用到 CIFAR-100 小型数据集上，完成迁移学习微调，并提交实验报告，内容包括

- 冻结策略、学习率设置、数据增强
- 训练/验证曲线(精度&损失)
- 最终模型性能与分析

01-03 没时间训练，代码写了，没有训练（cpu 内存不足）。三种冻结策略（fc_only / layer4 / layer3+）

待补充运行的任务

- 猫狗二分类实验代码的运行的代码编写（基础的 CNN）
- 比较解冻层数，对二分类任务的影响
- CIFAR-100 迁移学习

```
"""
猫狗二分类 - 超快速演示版
使用 ResNet18 + 500 张图片
"""

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import models, transforms
from PIL import Image
import os
import glob
from sklearn.model_selection import train_test_split
import numpy as np
import matplotlib.pyplot as plt

class CatDogDataset(Dataset):
```

```

def __init__(self, file_paths, transform=None):
    self.file_paths = file_paths
    self.transform = transform
def __len__(self):
    return len(self.file_paths)
def __getitem__(self, idx):
    img_path = self.file_paths[idx]
    image = Image.open(img_path).convert('RGB')
    filename = os.path.basename(img_path)
    label = 0 if filename.lower().startswith('cat') else 1
    if self.transform:
        image = self.transform(image)
    return image, label

# 数据预处理 (简化版)
train_transform = transforms.Compose([
    transforms.Resize(128),
    transforms.CenterCrop(128),
    transforms.RandomHorizontalFlip(),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

val_transform = transforms.Compose([
    transforms.Resize(128),
    transforms.CenterCrop(128),
    transforms.ToTensor(),
    transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

# 只用 500 张图片演示
data_dir = r'C:\TZ\Code\week11&12 GAN\dogs-vs-cats\train\train'
all_files = glob.glob(os.path.join(data_dir, '*.jpg'))
np.random.seed(42)
np.random.shuffle(all_files)
sample_files = all_files[:500] # 只用 500 张

train_files, val_files = train_test_split(sample_files, test_size=0.2,
random_state=42)
train_dataset = CatDogDataset(train_files, transform=train_transform)
val_dataset = CatDogDataset(val_files, transform=val_transform)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

```

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True,
num_workers=2)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False,
num_workers=2)

print(f"演示模式 - 数据集: {len(sample_files)} 张")
print(f"训练集: {len(train_dataset)} 张, 验证集: {len(val_dataset)} 张")
print(f"设备: {device}")

# 使用 ResNet18 (比 ResNet50 快很多)
def get_model(strategy):
    model = models.resnet18(pretrained=True)
    for param in model.parameters():
        param.requires_grad = False
    if strategy == 'layer4':
        for param in model.layer4.parameters():
            param.requires_grad = True
    elif strategy == 'unfrozen':
        for param in model.parameters():
            param.requires_grad = True
    model.fc = nn.Linear(model.fc.in_features, 2)
    return model.to(device)

def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss, correct, total = 0.0, 0, 0
    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
    return running_loss / len(loader), 100. * correct / total

def validate(model, loader, criterion, device):
    model.eval()
    running_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for images, labels in loader:

```

```

        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
    return running_loss / len(loader), 100. * correct / total

# 训练三种策略
strategies = ['fc_only', 'layer4', 'unfrozen']
results = {}

for strategy in strategies:
    print(f"\n{'='*50}")
    print(f"策略: {strategy}")
    print(f"{'='*50}")

    model = get_model(strategy)
    if strategy == 'fc_only':
        optimizer = optim.Adam(model.fc.parameters(), lr=1e-3)
    elif strategy == 'layer4':
        optimizer = optim.Adam([
            {'params': model.layer4.parameters(), 'lr': 1e-4},
            {'params': model.fc.parameters(), 'lr': 1e-3}
        ])
    else:
        optimizer = optim.Adam([
            {'params': model.layer1.parameters(), 'lr': 1e-5},
            {'params': model.layer2.parameters(), 'lr': 1e-5},
            {'params': model.layer3.parameters(), 'lr': 1e-4},
            {'params': model.layer4.parameters(), 'lr': 1e-4},
            {'params': model.fc.parameters(), 'lr': 1e-3}
        ])

    criterion = nn.CrossEntropyLoss()
    history = {'train_loss': [], 'train_acc': [], 'val_loss': [],
              'val_acc': []}
    best_val_acc = 0

    for epoch in range(5): # 只训练 5 个 epoch
        train_loss, train_acc = train_one_epoch(model, train_loader,
        criterion, optimizer, device)

```

```

        val_loss, val_acc = validate(model, val_loader, criterion,
device)
        history['train_loss'].append(train_loss)
        history['train_acc'].append(train_acc)
        history['val_loss'].append(val_loss)
        history['val_acc'].append(val_acc)
        print(f'Epoch [{epoch+1}/5] Train: {train_acc:.1f}% Val:
{val_acc:.1f}%')
        if val_acc > best_val_acc:
            best_val_acc = val_acc
            torch.save(model.state_dict(), f'demo_{strategy}.pth')

    results[strategy] = history
    print(f'最佳验证准确率: {best_val_acc:.2f}%')

# 绘图
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
for strategy, history in results.items():
    axes[0, 0].plot(history['train_loss'], label=strategy, marker='o')
    axes[0, 1].plot(history['train_acc'], label=strategy, marker='o')
    axes[1, 0].plot(history['val_loss'], label=strategy, marker='o')
    axes[1, 1].plot(history['val_acc'], label=strategy, marker='o')

axes[0, 0].set_title('Training Loss')
axes[0, 1].set_title('Training Accuracy')
axes[1, 0].set_title('Validation Loss')
axes[1, 1].set_title('Validation Accuracy')
for ax in axes.flat:
    ax.legend(); ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.savefig('demo_comparison.png', dpi=150)
print("\n 曲线图已保存: demo_comparison.png")

print("\n" + "="*50)
print("演示完成! 结果汇总:")
for strategy, history in results.items():
    print(f" {strategy}: {max(history['val_acc']):.2f}%")
print("="*50)

```


迁移学习代码

```
"""
CIFAR-100 迁移学习实验
使用预训练 ResNet50 进行微调
"""

import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import models, datasets, transforms
from torch.utils.data import DataLoader
import numpy as np
import matplotlib.pyplot as plt
import json
from collections import defaultdict
import time

# ===== 数据增强策略 =====
def get_data_transforms():
    """
    CIFAR-100 数据增强
    - 训练集：随机裁剪、翻转、颜色增强
    - 验证集：标准化处理
    """
    train_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.RandomResizedCrop(224),
        transforms.RandomHorizontalFlip(p=0.5),
        transforms.RandomRotation(15),
        transforms.ColorJitter(brightness=0.2, contrast=0.2,
saturation=0.2, hue=0.1),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
    ])

    val_transform = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224,
0.225])
    ])
```

```

        return train_transform, val_transform

# ===== 加载 CIFAR-100 数据集 =====
def load_cifar100(data_path='./data', batch_size=128):
    device = torch.device('cuda' if torch.cuda.is_available() else
                           'cpu')
    train_transform, val_transform = get_data_transforms()

    # 加载训练集并划分验证集
    train_dataset = datasets.CIFAR100(root=data_path, train=True,
                                       download=True, transform=train_transform)
    val_dataset = datasets.CIFAR100(root=data_path, train=True,
                                     download=True, transform=val_transform)

    # 手动划分训练集和验证集 (90% / 10%)
    train_size = int(0.9 * len(train_dataset))
    val_size = len(train_dataset) - train_size
    train_dataset, _ = torch.utils.data.random_split(train_dataset,
                                                      [train_size, val_size])
    _, val_dataset = torch.utils.data.random_split(val_dataset,
                                                    [train_size, val_size])

    # 测试集
    test_dataset = datasets.CIFAR100(root=data_path, train=False,
                                      download=True, transform=val_transform)

    train_loader = DataLoader(train_dataset, batch_size=batch_size,
                              shuffle=True, num_workers=4, pin_memory=(device.type=='cuda'))
    val_loader = DataLoader(val_dataset, batch_size=batch_size,
                            shuffle=False, num_workers=4, pin_memory=(device.type=='cuda'))
    test_loader = DataLoader(test_dataset, batch_size=batch_size,
                              shuffle=False, num_workers=4, pin_memory=(device.type=='cuda'))

    print(f"CIFAR-100 数据加载完成:")
    print(f" 训练集: {len(train_dataset)} 张")
    print(f" 验证集: {len(val_dataset)} 张")
    print(f" 测试集: {len(test_dataset)} 张")

    return train_loader, val_loader, test_loader

# ===== 冻结策略配置 =====
def apply_freeze_strategy(model, strategy):
    """
    冻结策略:

```



```

- 'fc_only': 只训练分类头 (100 类)
- 'layer4': 解冻 layer4 + fc
- 'layer3+': 解冻 layer3 + layer4 + fc
- 'unfrozen': 全部解冻
"""

# 先冻结所有层
for param in model.parameters():
    param.requires_grad = False

if strategy == 'fc_only':
    pass # 保持全部冻结, 只训练 fc
elif strategy == 'layer4':
    for param in model.layer4.parameters():
        param.requires_grad = True
elif strategy == 'layer3+':
    for param in model.layer3.parameters():
        param.requires_grad = True
    for param in model.layer4.parameters():
        param.requires_grad = True
elif strategy == 'unfrozen':
    for param in model.parameters():
        param.requires_grad = True

return model

# ===== 学习率设置策略 =====
def get_optimizer_and_scheduler(model, strategy, epochs):
    """
    分层学习率策略 + 余弦退火调度器
    """
    if strategy == 'fc_only':
        optimizer = optim.Adam(model.fc.parameters(), lr=1e-3)
    elif strategy == 'layer4':
        optimizer = optim.Adam([
            {'params': model.layer4.parameters(), 'lr': 1e-4},
            {'params': model.fc.parameters(), 'lr': 1e-3}
        ])
    elif strategy == 'layer3+':
        optimizer = optim.Adam([
            {'params': model.layer3.parameters(), 'lr': 5e-5},
            {'params': model.layer4.parameters(), 'lr': 1e-4},
            {'params': model.fc.parameters(), 'lr': 1e-3}
        ])
    else: # unfrozen

```

```

optimizer = optim.Adam([
    {'params': model.conv1.parameters(), 'lr': 1e-6},
    {'params': model.bn1.parameters(), 'lr': 1e-6},
    {'params': model.layer1.parameters(), 'lr': 1e-6},
    {'params': model.layer2.parameters(), 'lr': 1e-5},
    {'params': model.layer3.parameters(), 'lr': 1e-5},
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model.fc.parameters(), 'lr': 1e-3}
])

scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=epochs)
return optimizer, scheduler

# ===== 创建模型 =====
def create_model(strategy):
    device = torch.device('cuda' if torch.cuda.is_available() else
'cpu')
    model = models.resnet50(pretrained=True)

    # 应用冻结策略
    model = apply_freeze_strategy(model, strategy)

    # 替换分类头 (100 类)
    model.fc = nn.Linear(model.fc.in_features, 100)
    model = model.to(device)

    return model, device

# ===== 训练函数 =====
def train_one_epoch(model, loader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

```

```

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    return running_loss / len(loader), 100. * correct / total

# ===== 验证函数 =====
def validate(model, loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)

            running_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    return running_loss / len(loader), 100. * correct / total

# ===== 完整训练流程 =====
def train_cifar100(strategy='fc_only', epochs=30, batch_size=128):
    print(f"\n{'='*70}")
    print(f"CIFAR-100 迁移学习 - 策略: {strategy}")
    print(f"{'='*70}")

    # 加载数据
    train_loader, val_loader, test_loader =
load_cifar100(batch_size=batch_size)

    # 创建模型
    model, device = create_model(strategy)
    criterion = nn.CrossEntropyLoss()
    optimizer, scheduler = get_optimizer_and_scheduler(model, strategy,
epochs)

```

```

# 记录历史
history = {
    'train_loss': [], 'train_acc': [],
    'val_loss': [], 'val_acc': [],
    'lr': []
}

best_val_acc = 0
start_time = time.time()

for epoch in range(epochs):
    train_loss, train_acc = train_one_epoch(model, train_loader,
criterion, optimizer, device)
    val_loss, val_acc = validate(model, val_loader, criterion,
device)

    scheduler.step()
    current_lr = optimizer.param_groups[0]['lr']

    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    history['lr'].append(current_lr)

    print(f'Epoch [{epoch+1}/{epochs}] LR: {current_lr:.6f}')
    print(f'  Train Loss: {train_loss:.4f}, Acc: {train_acc:.2f}%')
    print(f'  Val   Loss: {val_loss:.4f}, Acc: {val_acc:.2f}%')

    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(),
f'cifar100_best_{strategy}.pth')
        print(f' *** Best model saved! (Val Acc: {val_acc:.2f}%
***')

# 测试集评估
model.load_state_dict(torch.load(f'cifar100_best_{strategy}.pth'))
test_loss, test_acc = validate(model, test_loader, criterion,
device)

elapsed_time = time.time() - start_time
print(f"\n{strategy} 训练完成!")
print(f" 最佳验证准确率: {best_val_acc:.2f}%")

```

```

print(f" 测试集准确率: {test_acc:.2f}%")
print(f" 训练时间: {elapsed_time/60:.1f} 分钟")

return history, best_val_acc, test_acc

# ===== 绘制训练曲线 =====
def plot_training_curves(history, strategy):
    fig, axes = plt.subplots(1, 3, figsize=(15, 4))

    # 损失曲线
    axes[0].plot(history['train_loss'], label='Train Loss', marker='o',
markersize=3)
    axes[0].plot(history['val_loss'], label='Val Loss', marker='s',
markersize=3)
    axes[0].set_title(f'{strategy} - Loss Curve')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Loss')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # 精度曲线
    axes[1].plot(history['train_acc'], label='Train Acc', marker='o',
markersize=3)
    axes[1].plot(history['val_acc'], label='Val Acc', marker='s',
markersize=3)
    axes[1].set_title(f'{strategy} - Accuracy Curve')
    axes[1].set_xlabel('Epoch')
    axes[1].set_ylabel('Accuracy (%)')
    axes[1].legend()
    axes[1].grid(True, alpha=0.3)

    # 学习率曲线
    axes[2].plot(history['lr'], label='Learning Rate', marker='o',
markersize=3, color='green')
    axes[2].set_title(f'{strategy} - Learning Rate Schedule')
    axes[2].set_xlabel('Epoch')
    axes[2].set_ylabel('Learning Rate')
    axes[2].legend()
    axes[2].grid(True, alpha=0.3)
    axes[2].set_yscale('log')

    plt.tight_layout()
    plt.savefig(f'cifar100_curves_{strategy}.png', dpi=150)
    print(f"训练曲线已保存: cifar100_curves_{strategy}.png")

```

```

# ===== 多策略对比 =====
def compare_strategies(strategies=['fc_only', 'layer4', 'layer3+'],
epochs=30):
    all_results = {}

    for strategy in strategies:
        history, best_val, test_acc = train_cifar100(strategy,
epochs=epochs)
        all_results[strategy] = {
            'history': history,
            'best_val_acc': best_val,
            'test_acc': test_acc
        }

        # 保存历史
        with open(f'cifar100_history_{strategy}.json', 'w') as f:
            json.dump({k: [float(x) if isinstance(x, (int, float)) else
x for k, v in history.items()]
                        for k, v in history.items()}), f, indent=2)

        # 绘制单独曲线
        plot_training_curves(history, strategy)

    # 绘制对比图
    plot_comparison(all_results)

    # 打印总结
    print("\n" + "="*70)
    print("各策略性能对比:")
    print("="*70)
    print(f"{'策略':<15} {'最佳验证准确率':<15} {'测试集准确率':<15}")
    print("-"*70)
    for strategy, results in all_results.items():
        print(f"{'strategy':<15} {'results['best_val_acc']':<15.2f}
{'results['test_acc']':<15.2f}")
        print("-"*70)

    # 保存汇总
    summary = {k: {'best_val_acc': float(v['best_val_acc']), 'test_acc':
float(v['test_acc'])}
                for k, v in all_results.items()}
    with open('cifar100_summary.json', 'w') as f:
        json.dump(summary, f, indent=2)

```

```

    return all_results

# ===== 策略对比图 =====
def plot_comparison(all_results):
    strategies = list(all_results.keys())

    fig, axes = plt.subplots(1, 2, figsize=(14, 5))

    # 验证精度对比
    for strategy in strategies:
        axes[0].plot(all_results[strategy]['history']['val_acc'],
                    label=strategy, marker='o', markersize=4)
    axes[0].set_title('Validation Accuracy Comparison')
    axes[0].set_xlabel('Epoch')
    axes[0].set_ylabel('Accuracy (%)')
    axes[0].legend()
    axes[0].grid(True, alpha=0.3)

    # 测试准确率柱状图
    test_accs = [all_results[s]['test_acc'] for s in strategies]
    bars = axes[1].bar(strategies, test_accs, color=['#1f77b4',
    '#ff7f0e', '#2ca02c'][:len(strategies)])
    axes[1].set_title('Test Accuracy Comparison')
    axes[1].set_ylabel('Accuracy (%)')
    axes[1].set_ylim(min(test_accs) - 2, max(test_accs) + 2)

    for bar, acc in zip(bars, test_accs):
        axes[1].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
0.5,
                    f'{acc:.2f}%', ha='center', fontsize=11)

    plt.tight_layout()
    plt.savefig('cifar100_comparison.png', dpi=150)
    print("\n 对比图已保存: cifar100_comparison.png")

# ===== 主程序 =====
if __name__ == '__main__':
    # 设置随机种子
    torch.manual_seed(42)
    np.random.seed(42)

    print("="*70)
    print("CIFAR-100 迁移学习实验")

```

```

print("="*70)
print("\n 实验设置:")
print("  - 模型: ResNet50 (预训练于 ImageNet)")
print("  - 数据集: CIFAR-100 (100 类, 45k 训练, 5k 测试)")
print("  - 数据增强: 随机裁剪、翻转、旋转、颜色抖动")
print("  - 学习率调度: Cosine Annealing")
print("  - 冻结策略: fc_only / layer4 / layer3+")
print("="*70)

# 运行对比实验
results = compare_strategies(
    strategies=['fc_only', 'layer4', 'layer3+'],
    epochs=30
)

print("\n 实验完成! 所有结果已保存。")

```

04. 迁移学习适用于解决什么样的问题？

- 解决源域与目标域之间在特征相似度、模型、结构上有类似之处的问题（围棋-象棋、学完 python 学 c++）

05. 如何理解“预训练”与“微调”？

- **预训练**：在大规模数据集上训练模型，学习通用的特征表示。
- **特点**：
 - a) **数据量**：非常大（如 ImageNet 的 1400 万图片、整个互联网文本）
 - b) **目标**：学习通用知识（边缘、纹理、语义等基础特征）
 - c) **成本**：高计算资源，时间长
 - d) **产出**：一个预训练模型权重
 - e) **例子**：在 ImageNet 上训练 ResNet，学会识别“什么是猫”、“什么是车”等 1000 类通用概念
- **微调**：将预训练模型迁移到特定任务上，用少量数据调整参数。
- **特点**：
 - a) **数据量**：相对较小（几百到几万样本）
 - b) **目标**：适配特定领域
 - c) **成本**：低计算资源，快
 - d) **方法**：冻结部分层 + 训练部分层，或全部层用小学习率训练
 - e) **例子**：用医疗 X 光片微调预训练的 ResNet，让它识别“肺炎”这种特定疾病。

06. 迁移学习中的源域和目标域有什么关系？

- 在特征空间的关系：可能相同可能不同（比如都是识别图片，有的黑白，有的彩色）
- 数据分布的关系：比如源域是白天街景图像；目标域是夜晚街景图像
- 07. 迁移学习中，使用什么方式“冻结”模型参数？
- # 方法 1: 设置 `requires_grad = False`

```
for param in model.parameters():
```

```
    param.requires_grad = False
```

```
# 替换分类头
```

```
model.fc = nn.Linear(model.fc.in_features, 2)
```

- # 方法 2: 优化器只传需要训练的参数

```
optimizer = torch.optim.Adam(model.fc.parameters(), lr=1e-3)
```

08. DANN（Domain-Adversarial Neural Networks）中梯度反转层（GRL）的作用是怎样的？

GRL 让特征提取器和领域分类器进行对抗：领域分类器努力区分域，特征提取器被迫抹去域信息，最终学到跨域通用的特征。

09. 如何理解 GAN 中生成器与判别器的作用？

判别器就是检察作用(D)，判别生成的与真实的是否接近的程度从而决定对生成器进行怎么样优化；生成器(G)是产生新样本的“函数”，产生的样本需要被判别器判断。G 和 D 互相对抗，互相进步。最终需要的是使用生成器 G 来生成一些“逼真”的内容，比如以假乱的人脸图片。

10. 通过 DANN 和 GAN 的学习，理解“对抗”的作用？

对抗就是两个目标相反的模块互相竞争，在博弈中达到平衡。

- GAN 里面是生成器 G 和判别器 D 的对抗。

对抗过程：

G 尽量生成逼真的样本

D 尽量识别出假样本

两者博弈，G 越来越强 → 能生成以假乱真的数据

- DANN 是特征提取器 E 和领域分类器 D 的对抗。

对抗过程：

D 努力识别样本来自哪个域

E 通过 GRL 收到反向梯度，被迫抹去域信息

两者博弈，E 越来越强 → 学到跨域通用特征

11. 分析视频中 GAN 训练过程中的生成器损失不降反生的可能原因，并尝试将模型架构进行优化，总结实验报告。

判别器 D 学得太快/太强，把 G 打趴了；相当于假钞制造者（G）和验钞机(D)能力不匹配，假钞制造者水平太低了，印钞机识别能力（D）太强了，达不到 GAN 中的“均衡对抗”的作用。

准备做的改进计划：

- 把优化器改成：optim.Adam(..., lr=0.0002, betas=(0.5, 0.999))
- **削弱 D 或加强 G**：例如 lr_D 调小（如 1e-4）、lr_G 调大，或 **每次更新 D 1 次，更新 G 2 次**
- 用 BCEWithLogitsLoss 并移除 Discriminator 里最后的 Sigmoid()
- 加一点 **label smoothing**（如 real label=0.9）或对 D 输入加少量噪声，避免 D 过快饱和

Week 12 强化学习&元学习&图模型 思考题

01. 强化学习与监督学习的本质区别在哪里？

- 有监督学习本质是授人以鱼，通过大量有标签数据的学习（标签用来算 loss），来解决某类特定任务。
- 强化学习是没有标签的，自然就没有 loss 函数，而取而代之的是通过奖励惩罚策略来指导调整学习行为。

02. 现实生活中哪些问题适合用强化学习解决？

- 需要连续多步决策：不是一次性预测/分类，而是“做一连串动作”
- 结果有延迟：做了动作，收益/损失可能过一段时间才体现（炒股）
- 环境会反馈：系统会因为用户的选择而改变（用户决策会“影响世界”）
- 规则难写或太复杂：靠手写策略很难覆盖所有情况
- 可以安全地模拟/离线回放：能在仿真里试错，或有足够历史数据做离线 RL

03. 对比孪生网络与原型网络

孪生网络是学一个相似度函数，适用于“是否同一个人/同一类”的验证问题；原型网络是每个类用一个“原型中心”代表

04.什么是 Zero-shot Learning、以及 Few-shot Learning？

- Zero-shot Learning 是新类别只有很少标注样本（比如每类 1、5、10 张），模型仍能快速学会区分这些新类别。
- Zero-shot Learning 是新类别一个标注样本也没有（0-shot），仍能识别

05.在 GNN 的基础上了解什么是 GCN（图卷积网络）和 GAT（图注意力网络）

- GCN 的核心就是用“归一化的邻接关系”做邻居的加权平均，让每个节点从邻居（包含自己）聚合信息。
- 临近节点不是平均的权重，而是用“注意力权重”决定谁更重要。也就是说，GCN 的聚合权重来自图结构（归一化度），GAT 的权重是学出来的

06.（必答题）12 周课程即将结束，请将你对课程的感受描述一下，以方便课程的不断完善。可以从课程内容、讲授方式、作业方式等任何你认为有感触的各个方面提出你宝贵的意见~~

答：总体而言，这门课程给我的体验非常棒。最大的特色在于强调实践——几乎每个关键知识点都安排了亲手编码的环节，这种“通过做来学”的方式让我能迅速理解原理，印象也更加深刻。全程跟下来收获满满，学习体验十分顺畅。

如果非要说一点建议的话，希望在每个主要模块（比如 CNN）讲解结束后，可以适当贴出一些该领域当前的前沿方向或经典论文/模型名称（例如 CLIP、Vision Transformer 等），不需要详细展开，只是作为“知识延伸”或“兴趣指引”，供学有余力的同学自行探索。这样既能开阔视野，也能帮助大家了解技术演进脉络，激发进一步学习的兴趣。

感谢老师的悉心指导与用心设计，辛苦了！